



4. Vertex AI 소개 및 개요

```
1 !pip install -q datasets torchserve torch-model-archiver torch-workflow-archiver nvgpu
```

```
Preparing metadata (setup.py) ... done
```

```
42.2/42.2 MB 7.1 MB/s eta 0:00:00
```

```
52.8/52.8 kB 2.1 MB/s eta 0:00:00
```

```
Building wheel for nvgpu (setup.py) ... done
```

```
1 import torch
2 import torch.nn as nn
3 import torch.optim as optim
4 import torch.nn.functional as F
5 import seaborn as sns
6 from torch.optim import AdamW
7
8 from torch.utils.data import DataLoader
9 from transformers import BatchEncoding, BertTokenizer, BertForSequenceClassification
10
11 from sklearn.metrics import confusion_matrix
12 from datasets import load_dataset
13 from tqdm import tqdm
14 from typing import TypedDict
15 import matplotlib.pyplot as plt
```

```
1 dataset = load_dataset("ag_news")
2 tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")
3 model = BertForSequenceClassification.from_pretrained("bert-base-uncased", num_labels=4)
4 optimizer = AdamW(model.parameters(), lr=5e-5)
5 criterion = torch.nn.CrossEntropyLoss()
6
7 class DatasetItem(TypedDict):
8     text: str
9     label: str
10
11
12 def preprocess_data(dataset_item: DatasetItem) -> dict[str, torch.Tensor]:
13     return tokenizer(dataset_item["text"], truncation=True, padding="max_length", return_tensors="pt")
14
15
16 train_dataset = dataset["train"].select(range(1200)).map(preprocess_data, batched=True)
17 test_dataset = dataset["test"].select(range(800)).map(preprocess_data, batched=True)
18
19 train_dataset.set_format("torch", columns=["input_ids", "attention_mask", "label"])
20 test_dataset.set_format("torch", columns=["input_ids", "attention_mask", "label"])
21
22 train_loader = DataLoader(train_dataset, batch_size=8, shuffle=True)
23 test_loader = DataLoader(test_dataset, batch_size=8, shuffle=False)
```

```

/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:93: UserWarning:
The secret 'HF_TOKEN' does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/t)
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
warnings.warn(
README.md:      8.07k/? [00:00<00:00, 570kB/s]
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limit
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a
data/train-00000-of-00001.parquet: 100%          18.6M/18.6M [00:01<00:00, 53.2MB/s]

data/test-00000-of-00001.parquet: 100%          1.23M/1.23M [00:00<00:00, 3.54MB/s]

Generating train split: 100%                    120000/120000 [00:00<00:00, 201730.00 examples/s]

Generating test split: 100%                     7600/7600 [00:00<00:00, 84638.02 examples/s]

tokenizer_config.json: 100%                     48.0/48.0 [00:00<00:00, 2.05kB/s]

vocab.txt:      232k/? [00:00<00:00, 2.31MB/s]

tokenizer.json:      466k/? [00:00<00:00, 2.78MB/s]

config.json: 100%                    570/570 [00:00<00:00, 29.0kB/s]

model.safetensors: 100%                440M/440M [00:03<00:00, 669MB/s]

Loading weights: 100%                  199/199 [00:00<00:00, 303.30it/s, Materializing param=bert.pooler.dense.weight]
BertForSequenceClassification LOAD REPORT from: bert-base-uncased
Key                                     | Status |
+-----+-----+
cls.predictions.transform.LayerNorm.bias | UNEXPECTED |
cls.predictions.transform.dense.bias     | UNEXPECTED |
cls.seq_relationship.bias                 | UNEXPECTED |
cls.predictions.transform.dense.weight   | UNEXPECTED |
cls.predictions.bias                     | UNEXPECTED |
cls.predictions.transform.LayerNorm.weight | UNEXPECTED |
cls.seq_relationship.weight               | UNEXPECTED |
classifier.bias                           | MISSING   |
classifier.weight                         | MISSING   |

Notes:
- UNEXPECTED :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING :those params were newly initialized because missing from the checkpoint. Consider training on your data

Map: 100%          1200/1200 [00:02<00:00, 493.54 examples/s]

Map: 100%          800/800 [00:01<00:00, 558.28 examples/s]

```

```

1 device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
2 print(device)

```

cuda

```

1 model.to(device)
2
3 num_epochs = 3
4 losses: list[float] = []
5
6 for epoch in range(num_epochs):
7     model.train()
8     total_loss = 0
9     for batch in tqdm(train_loader, desc=f"Epoch {epoch + 1}"):
10         inputs = {key: batch[key].to(device) for key in batch}
11         labels = inputs.pop("label")
12         outputs = model(**inputs, labels=labels)
13         loss = outputs.loss
14         total_loss += loss.item()
15         losses.append(loss.item())
16
17     optimizer.zero_grad()
18     loss.backward()
19     optimizer.step()
20
21     average_loss = total_loss / len(train_loader)
22     print(f"Epoch {epoch + 1}, Average Loss: {average_loss}")

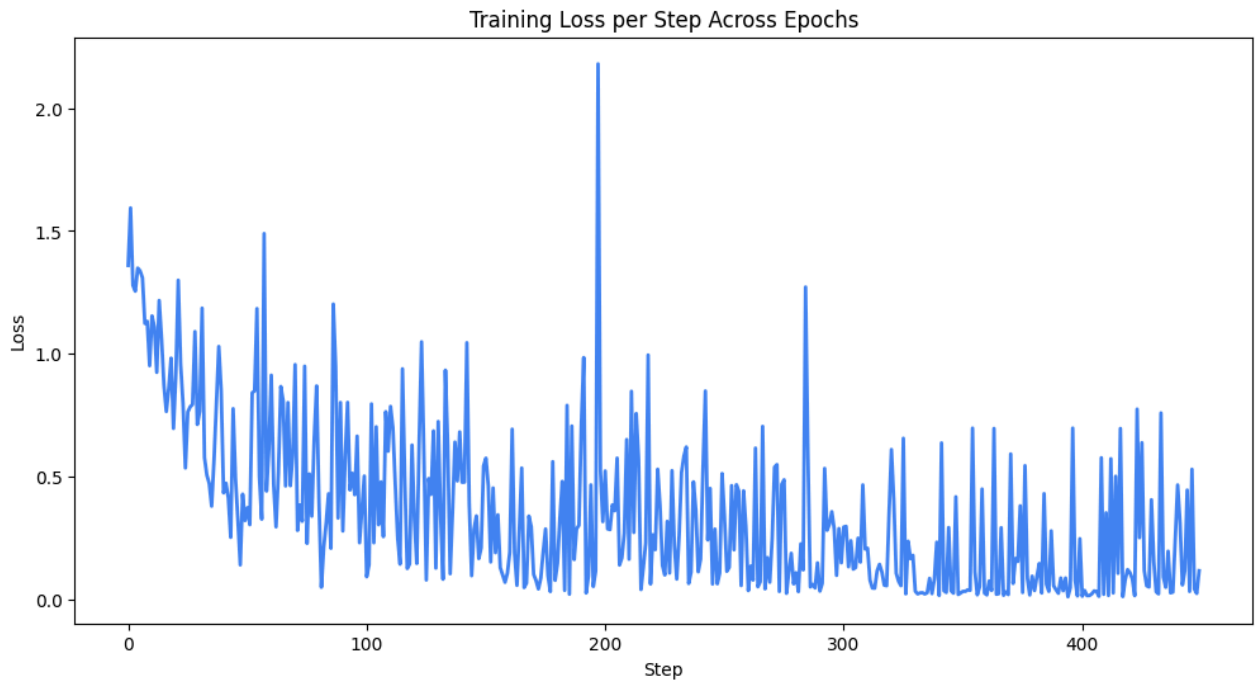
Epoch 1: 100%|██████████| 150/150 [01:45<00:00, 1.42it/s]
Epoch 1, Average Loss: 0.6195735860119264
Epoch 2: 100%|██████████| 150/150 [01:54<00:00, 1.31it/s]
Epoch 2, Average Loss: 0.3004276896889011
Epoch 3: 100%|██████████| 150/150 [01:54<00:00, 1.31it/s]Epoch 3, Average Loss: 0.16569822565962872

```

```

1 plt.figure(figsize=(12, 6))
2 plt.plot(losses, color="#4285f4", linewidth=2)
3 plt.xlabel("Step")
4 plt.ylabel("Loss")
5 plt.title("Training Loss per Step Across Epochs")
6 plt.show()

```



```

1 model.eval()
2 correct = 0
3 total = 0
4
5 with torch.no_grad():
6     for batch in tqdm(test_loader, desc="Evaluating"):
7         inputs = {key: batch[key].to(device) for key in batch}
8         labels = inputs.pop("label")
9         outputs = model(**inputs, labels=labels)
10        logits = outputs.logits
11        predicted_labels = torch.argmax(logits, dim=1)
12        correct += (predicted_labels == labels).sum().item()
13        total += labels.size(0)
14
15 accuracy = correct / total
16
17 print("")
18 print(f"Test Accuracy: {accuracy * 100:.2f}%")

```

Evaluating: 100%|██████████| 100/100 [00:24<00:00, 4.08it/s]
 Test Accuracy: 82.75%

original

Evaluating: 100%|██████████| 100/100 [00:21<00:00, 4.74it/s]
 Test Accuracy: 79.62%

```

1 test_input = "[Official] 'Legendary Coach Resigns → Appoints New Commander' Suwon Completes Coaching Staff.."
2 test_input_processed = tokenizer(test_input, truncation=True, padding="max_length", return_tensors="pt").to(
3 logits = model(**test_input_processed).logits
4 print(logits)
5 predicted_labels = torch.argmax(logits, dim=1)
6
7 labeling_mapper = ["world", "sports", "business", "sci/tech"]
8 print(labeling_mapper[predicted_labels[0]])

```

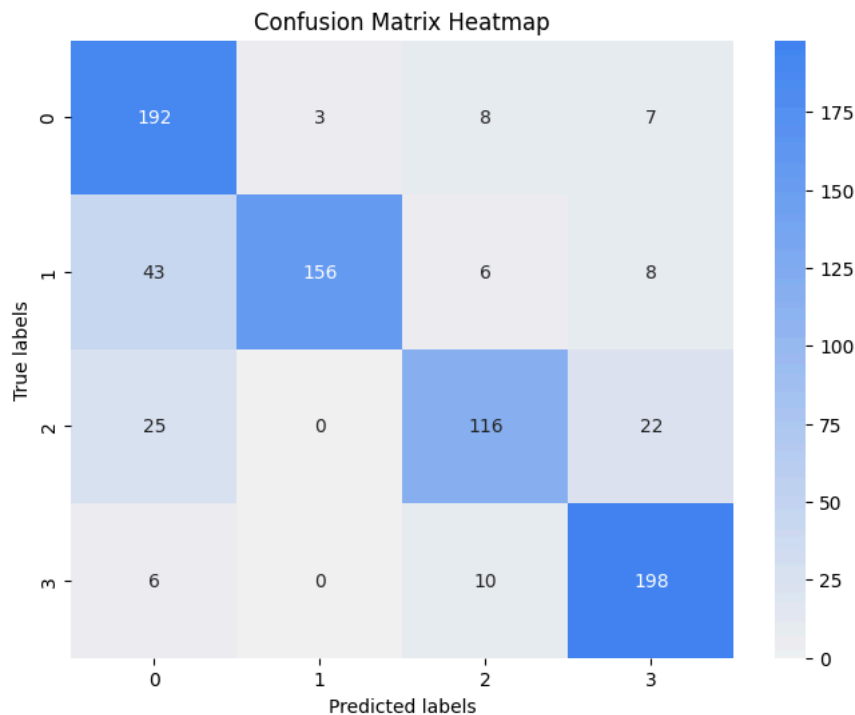
tensor([[2.2622, 1.9271, -1.9684, -2.2691]], device='cuda:0',
 grad_fn=<AddmmBackward0>)
 world

```
tensor([[ 2.3975,  2.9687, -2.3953, -3.2521]], device='cuda:0',
        grad_fn=<AddmmBackward0>)
sports
```

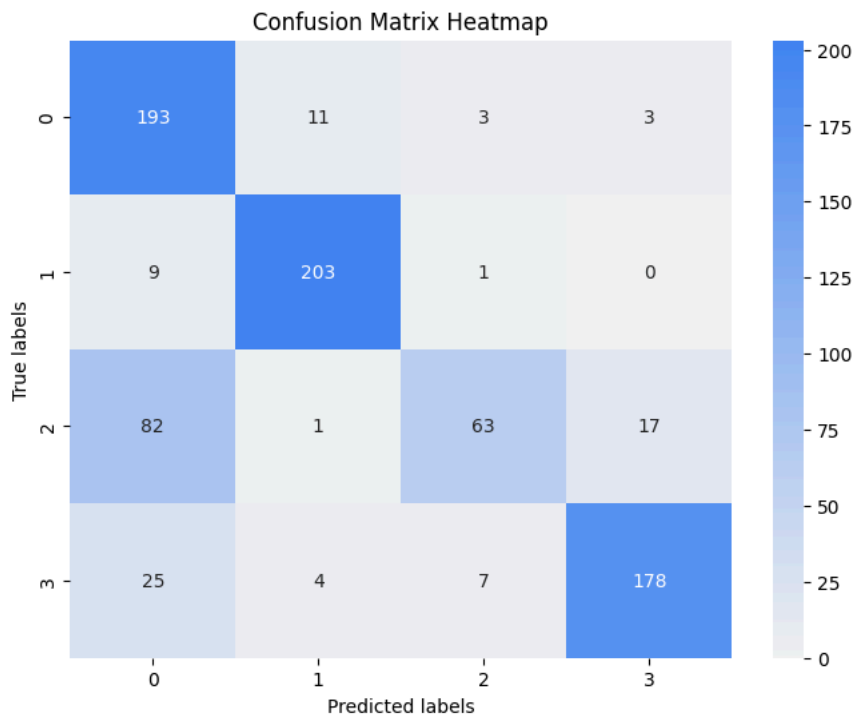
```
1 all_predictions: list[int] = []
2 all_labels: list[int] = []
3
4 with torch.no_grad():
5     for batch in tqdm(test_loader, desc="Evaluating"):
6         inputs = {key: batch[key].to(device) for key in batch}
7         labels = inputs.pop("label")
8         outputs = model(**inputs)
9         logits = outputs.logits
10        predicted_labels = torch.argmax(logits, dim=1)
11
12        all_predictions.extend(predicted_labels.cpu().numpy())
13        all_labels.extend(labels.cpu().numpy())
```

Evaluating: 100%|██████████| 100/100 [00:24<00:00, 4.13it/s]

```
1 conf_matrix = confusion_matrix(all_labels, all_predictions)
2 plt.figure(figsize=(8, 6))
3 sns.heatmap(conf_matrix, annot=True, fmt="g", cmap=sns.light_palette("#4285f4", as_cmap=True))
4 plt.xlabel("Predicted labels")
5 plt.ylabel("True labels")
6 plt.title("Confusion Matrix Heatmap")
7 plt.show()
```



원본



▼ TorchServe Archive

```
1 # 모델 체크포인트 저장.
2 model_save_path = "model.pth"
3 torch.save(model.state_dict(), model_save_path)
```

```
1 %%writefile handler.py
2 import json
3 import logging
4
5 import torch
6 from ts.context import Context
7 from ts.torch_handler.base_handler import BaseHandler
8 from transformers import BatchEncoding, BertTokenizer, BertForSequenceClassification
9
10 logging.basicConfig(level=logging.INFO)
11
12 class ModelHandler(BaseHandler):
13     def __init__(self):
14         self.initialized = False
15         self.tokenizer = None
16         self.model = None
17
18     def initialize(self, context: Context):
19         properties = context.system_properties
20         model_dir = properties.get("model_dir")
21         self.initialized = True
22         self.tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")
23         self.model = BertForSequenceClassification.from_pretrained("bert-base-uncased", num_labels=4)
24         model_path = model_dir + "/model.pth"
25         self.model.load_state_dict(torch.load(model_path))
26         self.model.to(torch.device("cuda" if torch.cuda.is_available() else "cpu"))
27         self.model.eval()
28
29     def preprocess(self, texts: list[str]) -> BatchEncoding:
30         logging.info("preprocess", texts)
31
32         inputs = self.tokenizer(texts, truncation=True, padding=True, max_length=512, return_tensors="pt")
33         device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
34         return inputs.to(device)
35
36     def inference(self, input_batch: BatchEncoding) -> torch.Tensor:
37         with torch.no_grad():
38             outputs = self.model(**input_batch)
39             logging.info("inference", outputs)
40             return outputs.logits
41
```

```

42 def postprocess(self, inference_output: torch.Tensor) -> list[dict[str, float]]:
43     logging.info("postprocess", inference_output)
44
45     probabilities = torch.nn.functional.softmax(inference_output, dim=1)
46     return [{"label": int(torch.argmax(prob)), "probability": float(prob.max())} for prob in probabilities]

```

Writing handler.py

```

1 # bert vocab 파일 (아티팩트)
2
3 !wget https://raw.githubusercontent.com/microsoft/SDNet/master/bert_vocab_files/bert-base-uncased-vocab.txt
4     -O bert-base-uncased-vocab.txt

```

```

--2026-04-22 01:06:31-- https://raw.githubusercontent.com/microsoft/SDNet/master/bert_vocab_files/bert-base-uncased-vocab.txt
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.108.133, 185.199.109.133, 185.199.110.133, 185.199.111.133
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.108.133|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 231508 (226K) [text/plain]
Saving to: 'bert-base-uncased-vocab.txt'

```

```
bert-base-uncased-v 100%[=====] 226.08K --.-KB/s in 0.009s
```

```
2026-04-22 01:06:32 (24.5 MB/s) - 'bert-base-uncased-vocab.txt' saved [231508/231508]
```

```

1 # Torch serve archiver의 최종 파일 저장 위치.
2 # .mar 확장자 파일을 생성하여 패키징 하게 된다.
3
4 !mkdir -p model-store

```

Register to Vertex AI

```

1 !torch-model-archiver \
2     --model-name model \
3     --version 1.0 \
4     --serialized-file model.pth \
5     --handler ./handler.py \
6     --extra-files "bert-base-uncased-vocab.txt" \
7     --export-path model-store \
8     -f

```

```

1 from google.colab import auth
2 auth.authenticate_user()

```

```

1 from google.cloud import storage, aiplatform
2 from google.oauth2 import service_account

```

```
1 credentials = service_account.Credentials.from_service_account_file("credentials.json")
```

```

1 PROJECT_ID = "aiffel-learning"
2 LOCATION = "asia-northeast3"
3 BUCKET_NAME = "github-action-cicd" # @param {type="string"}
4 MODEL_FILE_NAME = "model.mar" # @param {type="string"}

```

PROJECT_ID

aiffel-learning

LOCATION

asia-northeast3

BUCKET_NAME

github-action-cicd

MODEL_FILE_NAME

model.mar

[코드 숨기기](#)

```

1 storage_client = storage.Client(credentials=credentials)
2 bucket = storage_client.bucket(BUCKET_NAME)

```

```

1 def upload_blob(source_file_name: str, destination_blob_name: str) -> None:
2     blob = bucket.blob(destination_blob_name)
3     blob.upload_from_filename(source_file_name)
4     print(f"File {source_file_name} uploaded to {destination_blob_name}.")

```

```
1 upload_blob("model-store/model.mar", f"models/{MODEL_FILE_NAME}")
```

```
File model-store/model.mar uploaded to models/model.mar.
```

```

1 aiplatform.init(
2     project=PROJECT_ID,
3     location=LOCATION,

```

```
4 credentials=credentials,
5 )
```

```
1 model_path = f"gs://{BUCKET_NAME}/models"
2 registry_model = aiplatform.Model.upload(
3     display_name="AG News Classification",
4     artifact_uri=model_path,
5     serving_container_image_uri=f"asia-docker.pkg.dev/{PROJECT_ID}/github-action/trainer:fa06f56",
6     is_default_version=True,
7     version_aliases=["v1"],
8     version_description="A news category classification model",
9     serving_container_predict_route="/predictions/model",
10    serving_container_health_route="/ping",
11 )
```

```
# original
INFO:google.cloud.aiplatform.models:Creating Model
INFO:google.cloud.aiplatform.models:Create Model backing LR0: projects/634821526652/locations/asia-northeast3/model
INFO:google.cloud.aiplatform.models:Model created. Resource name: projects/634821526652/locations/asia-northeast3/m
INFO:google.cloud.aiplatform.models:To use this Model in another session:
INFO:google.cloud.aiplatform.models:model = aiplatform.Model('projects/634821526652/locations/asia-northeast3/model
```

```
1 DEPLOY_COMPUTE = "n1-standard-2"
2 DEPLOY_ACCELERATOR = "NVIDIA_TESLA_T4"
```

```
1 endpoint = aiplatform.Endpoint.create(
2     display_name="ag-news-category-classification",
3     project=PROJECT_ID,
4     location=LOCATION,
5 )
```

```
1 deployment = registry_model.deploy(
2     endpoint=endpoint,
3     machine_type=DEPLOY_COMPUTE,
4     min_replica_count=1,
5     max_replica_count=1,
6     accelerator_type=DEPLOY_ACCELERATOR,
7     accelerator_count=1,
8     traffic_percentage=100,
9     sync=True,
10 )
```

FailedPrecondition Traceback (most recent call last)
 /tmp/ipykernel_26963/1754561988.py in <cell line: 0>()

```
----> 1 deployment = registry_model.deploy(
2     endpoint=endpoint,
3     machine_type=DEPLOY_COMPUTE,
4     min_replica_count=1,
5     max_replica_count=1,
```

4 frames

/usr/local/lib/python3.12/dist-packages/google/api_core/future/polling.py in result(self, timeout, retry, polling)

```
259         # pylint: disable=raising-bad-type
260         # Pylint doesn't recognize that this is valid in this case.
--> 261         raise self._exception
262
263         return self._result
```

FailedPrecondition: 400 Model server exited unexpectedly. Model server logs can be found at [https://console.cloud.google.com/logs/viewer?project=81281014806&resource=aiplatform.googleapis.com%2FEndpoint&advancedFilter=resource.type%3D%22aiplatform.googleapis.com%2FEndpoint&advancedFilter=resource.type%3D%22aiplatform.googleapis.com%2FEndpoint](https://console.cloud.google.com/logs/viewer?project=81281014806&resource=aiplatform.googleapis.com%2FEndpoint&advancedFilter=resource.type%3D%22aiplatform.googleapis.com%2FEndpoint&advancedFilter=resource.type%3D%22aiplatform.googleapis.com%2FEndpoint&advancedFilter=resource.type%3D%22aiplatform.googleapis.com%2FEndpoint). 9: Model server exited unexpectedly. Model server logs can be found at <https://console.cloud.google.com/logs/viewer?project=81281014806&resource=aiplatform.googleapis.com%2FEndpoint&advancedFilter=resource.type%3D%22aiplatform.googleapis.com%2FEndpoint&advancedFilter=resource.type%3D%22aiplatform.googleapis.com%2FEndpoint>.

다음 단계: [오류 설명](#)

▼ 에러 발생

```
# original
INFO:google.cloud.aiplatform.models:Deploying model to Endpoint : projects/634821526652/locations/asia-northeast3/e
INFO:google.cloud.aiplatform.models:Deploy Endpoint model backing LR0: projects/634821526652/locations/asia-northea
INFO:google.cloud.aiplatform.models:Endpoint model deployed. Resource name: projects/634821526652/locations/asia-no
```

위와 같은 결과를 기대했는데 에러가 남.

```
-----
FailedPrecondition                                Traceback (most recent call last)
/tmp/ipykernel_26963/1754561988.py in <cell line: 0>()
----> 1 deployment = registry_model.deploy(
      2     endpoint=endpoint,
      3     machine_type=DEPLOY_COMPUTE,
      4     min_replica_count=1,
      5     max_replica_count=1,

4 frames
/usr/local/lib/python3.12/dist-packages/google/api_core/future/polling.py in result(self, timeout, retry, polling)
    259         # pylint: disable=raising-bad-type
    260         # Pylint doesn't recognize that this is valid in this case.
--> 261         raise self._exception
    262
    263         return self._result

FailedPrecondition: 400 Model server exited unexpectedly. Model server logs can be found at https://console.cloud.g
```

원인: trainer 이미지는 학습만 하고 종료되는 컨테이너.

Vertex AI는 배포 시 계속 떠있으면서 예측 요청을 받는 서빙 서버를 기대하는데,

trainer는 학습 후 그냥 꺼버리니까 "Model server exited unexpectedly" 에러가 나는 거라고 생각된다.

오르 책겨에 시간이 좀 더 걸리 거 가스니 기다 🙄

```
1 endpoint.predict(instances=[
2     "OpenAI releases AI video generator Sora to all customers"
3 ])
```

```
-----
_InactiveRpcError                                Traceback (most recent call last)
/usr/local/lib/python3.12/dist-packages/google/api_core/grpc_helpers.py in error_remapped_callable(*args,
**kwargs)
    54         try:
--> 55             return callable_(*args, **kwargs)
    56         except grpc.RpcError as exc:

-----
10 frames
_InactiveRpcError: <InactiveRpcError of RPC that terminated with:
  status = StatusCode.FAILED_PRECONDITION
  details = "Endpoint projects/81281014806/locations/asia-northeast3/endpoints/699335574753902592
misconfigured, "traffic_split" not set. Verify if any models are deployed to the endpoint and traffic split is
configured for them."
  debug_error_string = "UNKNOWN:Error received from peer ipv4:74.125.128.95:443 {grpc_status:9,
grpc_message:"Endpoint projects/81281014806/locations/asia-northeast3/endpoints/699335574753902592
misconfigured, \"traffic_split\" not set. Verify if any models are deployed to the endpoint and traffic split
is configured for them.\"}"
>

The above exception was the direct cause of the following exception:

FailedPrecondition                                Traceback (most recent call last)
/usr/local/lib/python3.12/dist-packages/google/api_core/grpc_helpers.py in error_remapped_callable(*args,
**kwargs)
    55         return callable_(*args, **kwargs)
    56         except grpc.RpcError as exc:
--> 57             raise exceptions.from_grpc_error(exc) from exc
    58
    59         return error_remapped_callable

FailedPrecondition: 400 Endpoint projects/81281014806/locations/asia-northeast3/endpoints/699335574753902592
```

다음 단계: [오류 설명](#)

```
Prediction(predictions=[{'label': 3.0, 'probability': 0.9921878576278687}], deployed_model_id='7880257010875236352', metadata=None,
model_version_id='1', model_resource_name='projects/634821526652/locations/asia-northeast3/models/351848118934831104',
explanations=None)
```